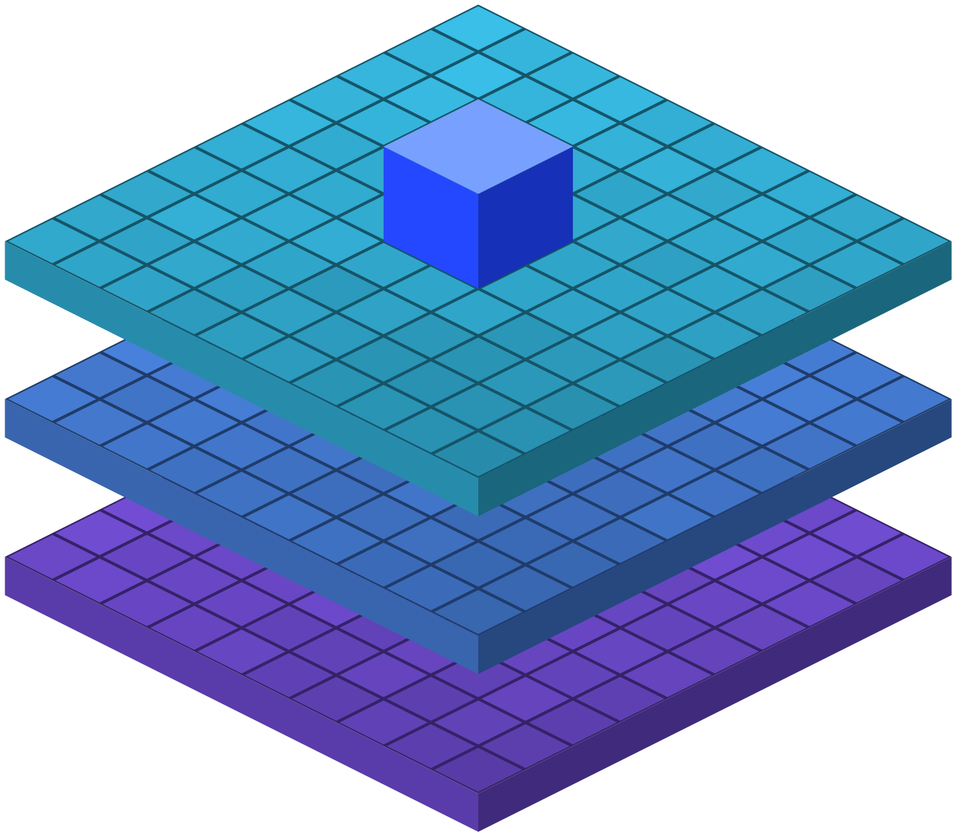
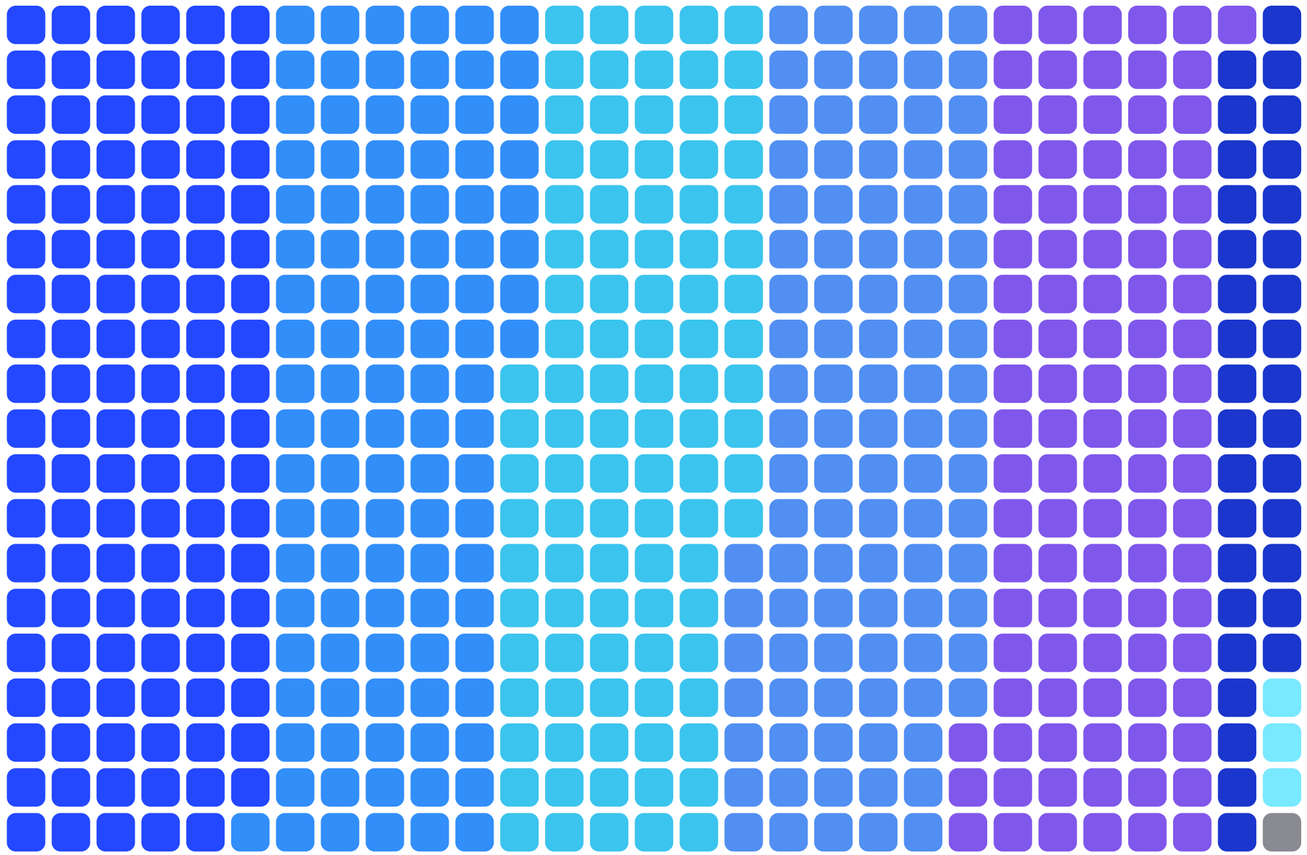
datum

Модель данных Capibara:
от корзины до выкупа



Хакатон, кейс «Модель маркетплейса»

550 759 строк *и все текстом*

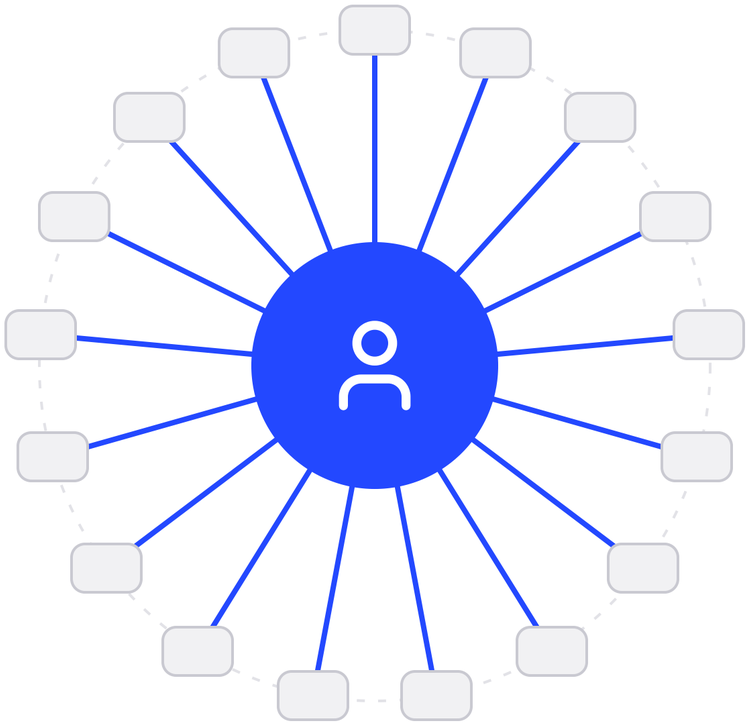


order_items	112 650
order_payments	103 886
orders	99 441
order_reviews	99 224
customers	99 441
products	32 951
sellers	3 095
category_translation	71

1 клетка = 1 000 строк. Даты, суммы и оценки приехали строками.

datum

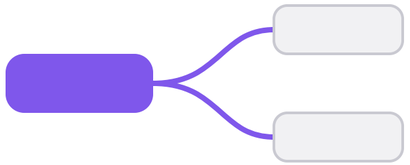
Анкета не равна *человеку*



до 17 анкет у одного человека

~~99 441~~ → **96 096**

`customer_id` → `customer_unique_id`



789

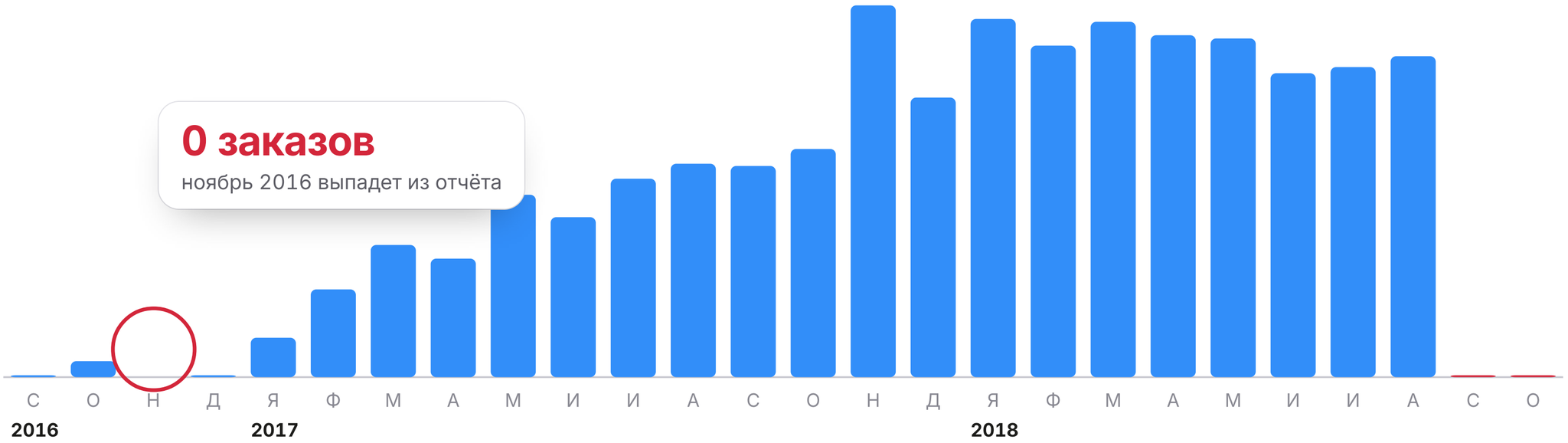
отзывов висят на 2 заказах

order_id 1 2 3

`order_item_id`

номер строки, а не ключ

Пропуски *и потери*



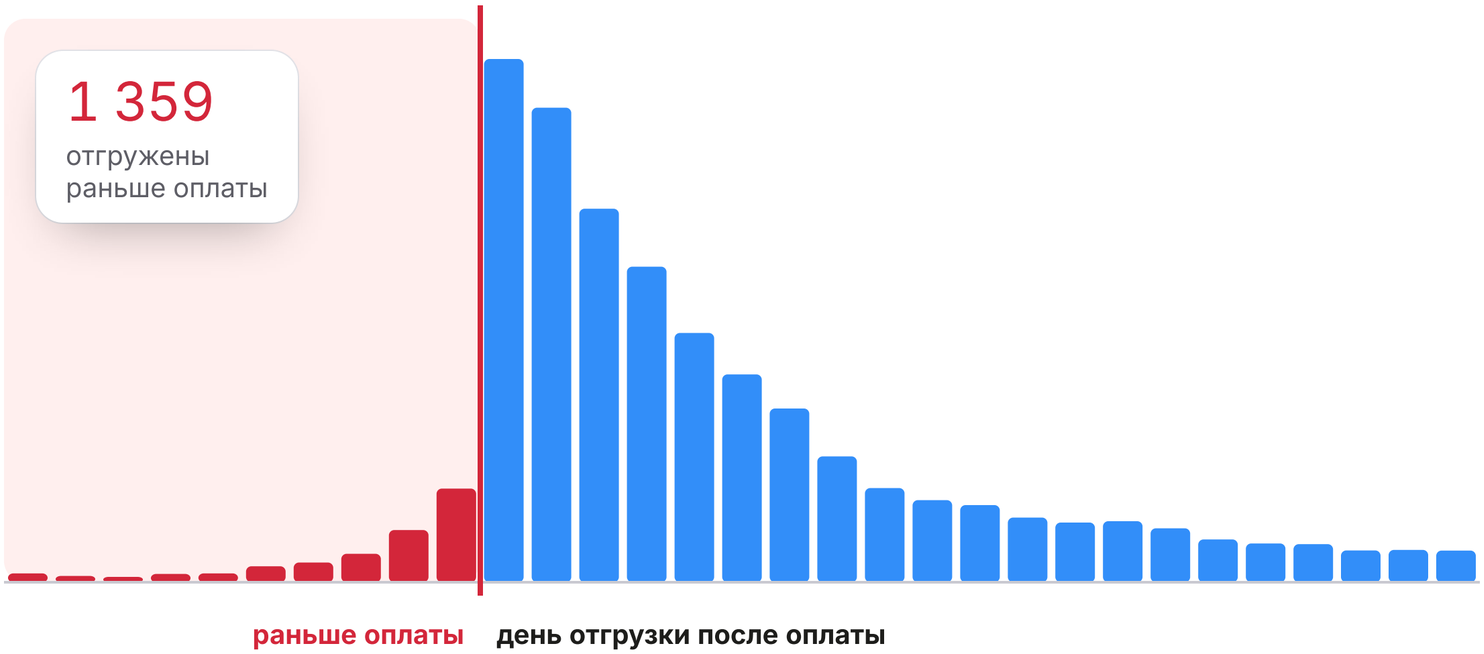
610
товаров без категории

775
заказов без позиций: unavailable и canceled

170 726,63
R$ выручки на товарах без категории

73 ≠ 71
категорий в товарах и в переводах

Хронология *и грязь*



23
вручены раньше отгрузки

6
отменённых с датой вручения

1 139
заказов: платежи не бьются с позициями

`auriflama/sp`

`rio de janeiro \ rio de janeiro`

`04482255`

611
написаний городов, индекс вместо города

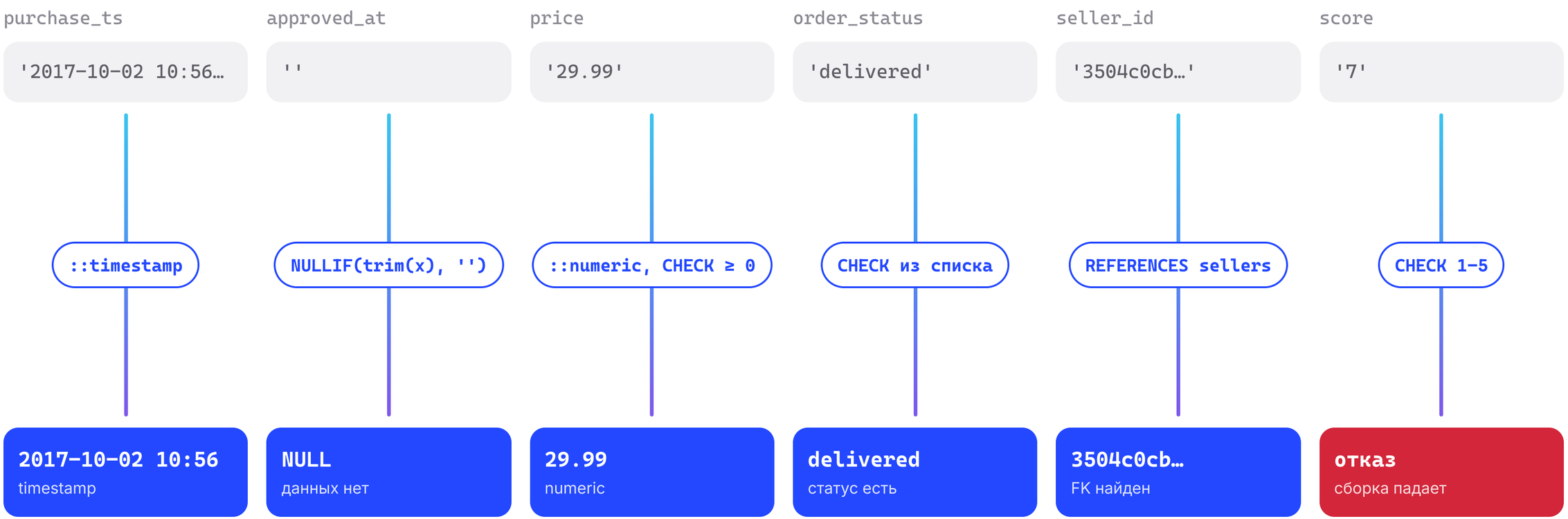
Вывод: источникам доверять нельзя. Проверки живут в модели, а не в голове аналитика.

datum

Как решили: *правила переноса*

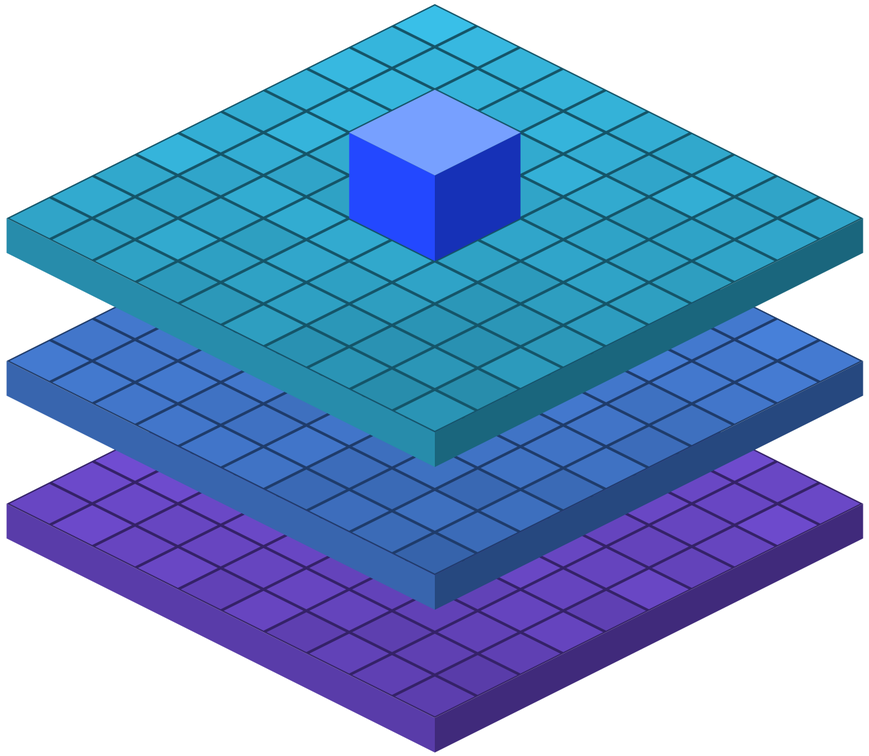
raw: всё text

core: типы, NULL, PK, FK, CHECK



Один скрипт `build_core.sql` , DROP SCHEMA и BEGIN … COMMIT: наполовину собранной модели не бывает

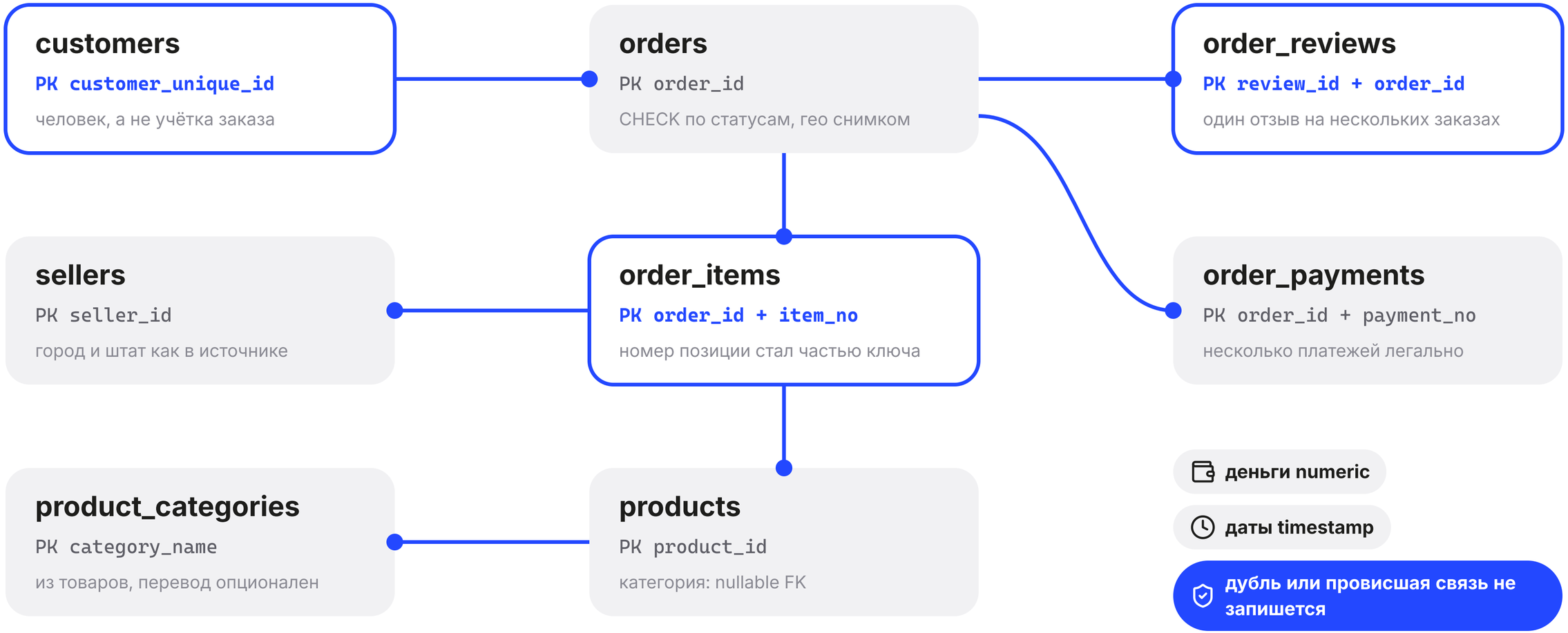
Три слоя, *как в нашем знаке*



- **ответы** считаются из core
- **mart** витрины под задачу: исполнение заказа
- **core** модель: типы, ключи, связи, дедупликация
- **raw** как приехало, только читаем

`build_core.sql` → `build_mart.sql` → `build_dashboard.py`

Схема core: *ключ это решение*



Почему *именно так*

	наивно		в модели	доля ошибки	решение
Покупатель	~~99 441~~	→	**96 096**		customer_unique_id, повторных 3.12% вместо 0
География	~~адрес сейчас~~	→	**адрес в заказе**		снимок: у 250 человек адрес менялся
Категории	~~71~~	→	**74**		справочник из товаров, перевод опционален
Без категории	~~13 050 771.48~~	→	**13 221 498.11**		отдельная строка, сумма сходится
Заказы в отчётах	~~98 666~~	→	**99 441**		LEFT JOIN, где пары может не быть

Ничего *не потеряли*



RAISE EXCEPTION витрины: строки, пары заказ и продавец, деньги. Расхождение валит сборку

Витрины: *один SELECT без join*



`order_fulfillment`

этапы в днях, on_time, суммы, последний отзыв.
Сломанная хронология в NULL

99 441

1 строка = 1 заказ

`seller_shipment`

срок отгрузки пары и флаг «успел ли»

100 010

1 строка = заказ × продавец

⚠ **`is_single_seller`**: дата отгрузки в источнике одна на заказ, поэтому рейтинг продавца честен только для заказов с одним продавцом.

Блок A: *все пять контролей сошлись*

13 221 498,11

выручка за 24 месяца, R$, без доставки



226 987.93

топ-продавец из Гуарибы: от позиций, а не от заказов

159.88

средний чек: заказ, потом среднее

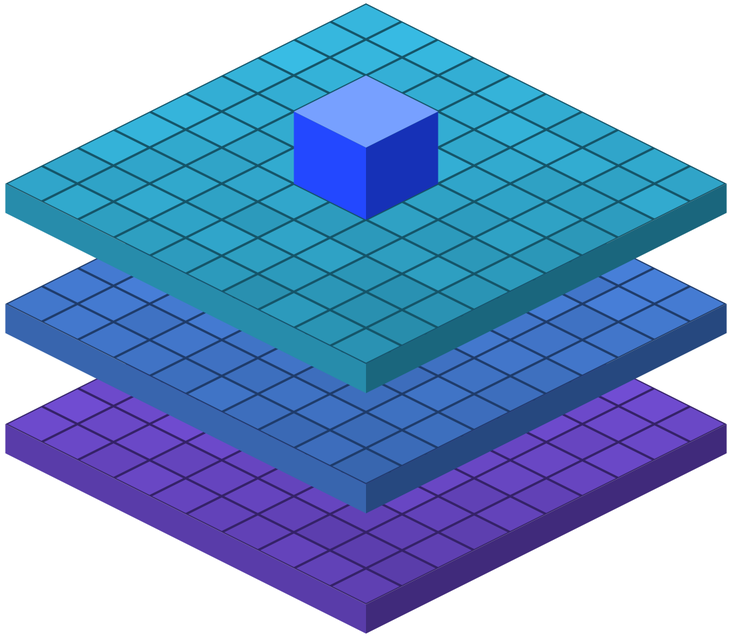
96 096 3.12%

покупателей по customer_unique_id

74

категории с «(без категории)»

datum

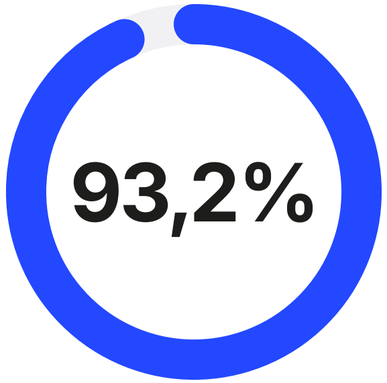


Готовы к вашим *вопросам*

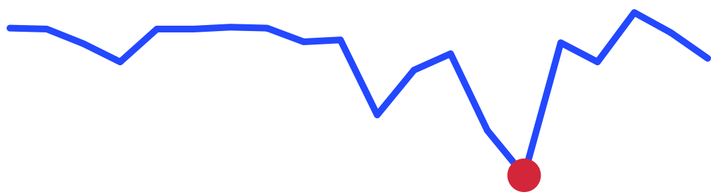
Одна модель, одна цифра для всех

datum

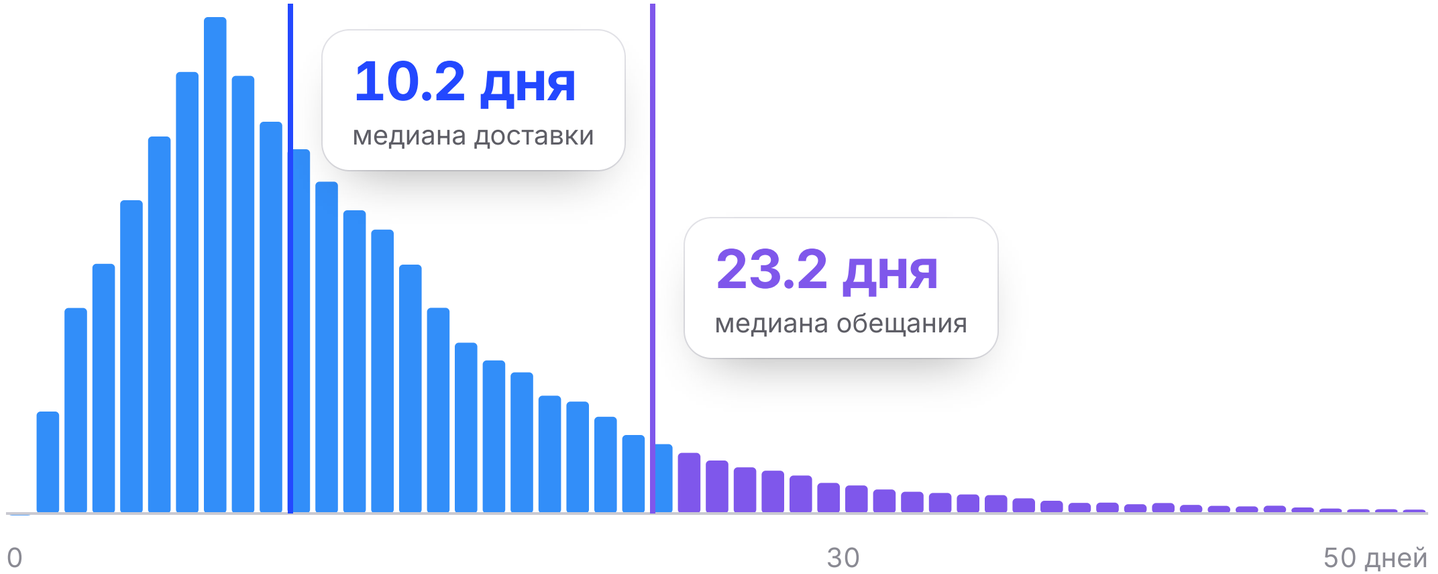
Дашборд *исполнения заказа*



вручено в обещанный срок



март 2018: 81%



1.8 / 7.1
дни продавца и перевозчика

1 107
застряли дольше обещания

90.9%
отгрузок продавцов в срок

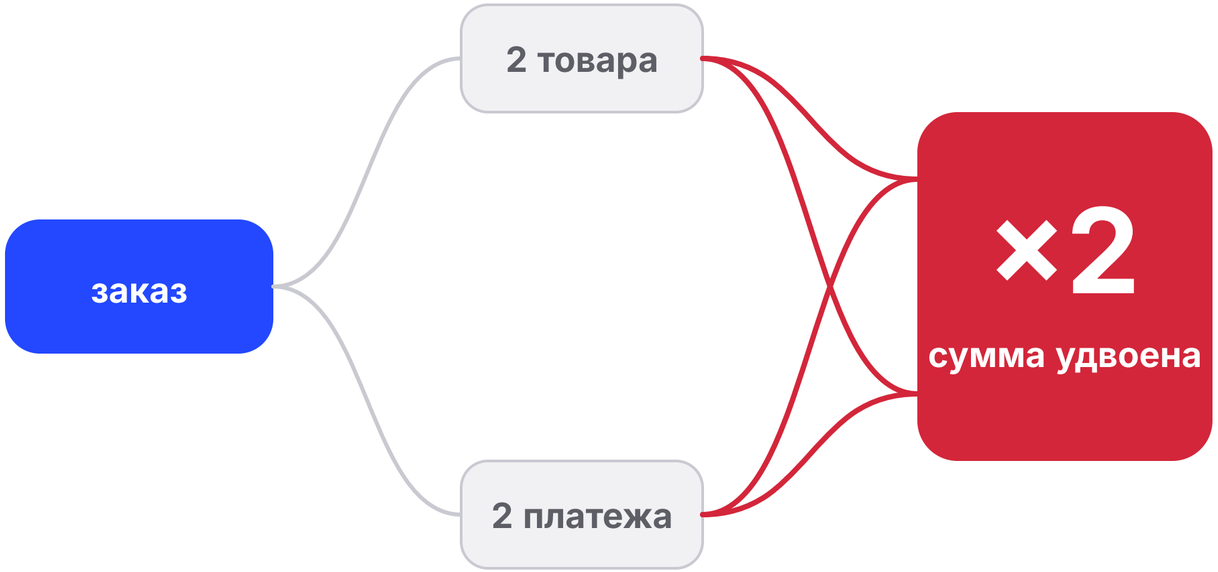
П1

datum

Агент считает *тем же SQL*



Один заказ, *четыре строки*



средний чек по заказам

159,88

среднее по строкам

139,97

−12.5%

и оба числа выглядят правдоподобно

Чем выше слой, *тем короче план*

Топ продавцов из **core** — 2 join

```
Limit 10
Sort: top-N по выручке
  GroupAggregate: 2 970 продавцов
    Hash Join: seller_id
      Hash Join: order_id
        Seq Scan items: 112 650
        Seq Scan orders: delivered
```

OTD по месяцам из **mart** — 0 join

```
GroupAggregate: 23 месяца
Sort: месяц
  Seq Scan order_fulfillment
```

1 проход

on_time и дни уже лежат в витрине

Блок Б: *места уже готовы*

6
вопрос

7
вопрос

8
вопрос

9
вопрос

10
вопрос

вброс
упрётся в типы, FK и CHECK

0
изменений схемы под новый вопрос

1
новый SELECT на вопрос

2
витрины уже хранят этапы и флаги